

AI ASS 15.4

Name:B.Sanjana

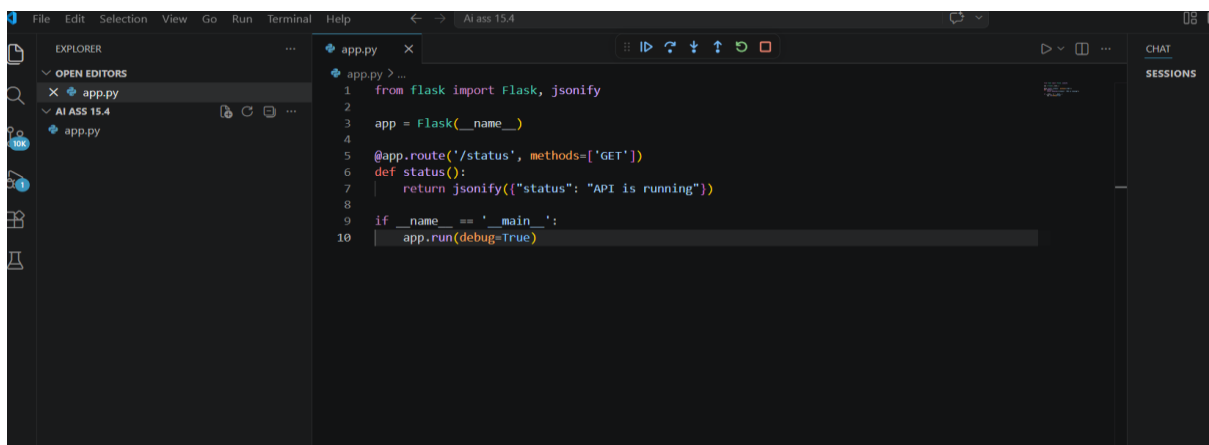
H.T.NO:2303A52306

Batch:43

Task1:

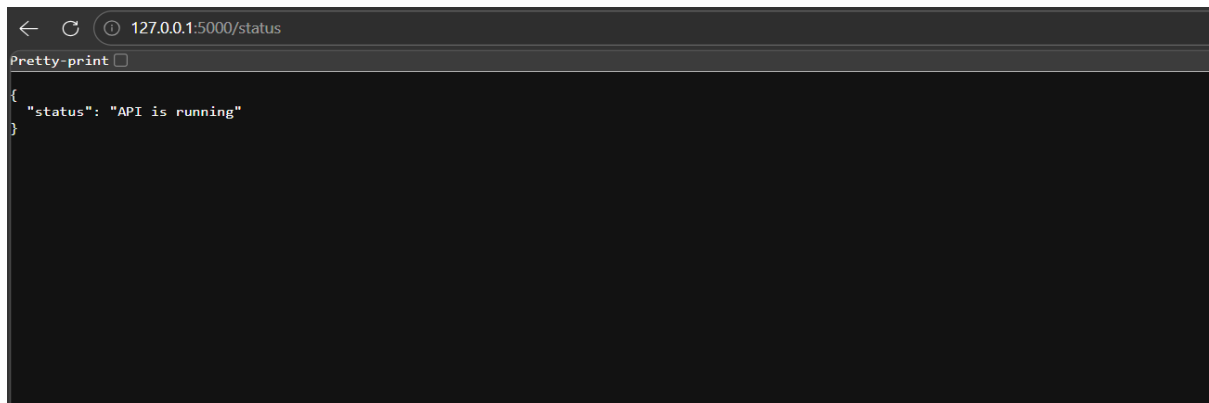
Prompt

- 1.Develop a basic Flask application for a service management platform.
- 2.Create a root endpoint /status that returns a JSON response confirming the API is live.
- 3.Run the server locally.
- 4.Verify the response using a browser or API testing tool (e.g., curl, Postman).

A screenshot of a code editor interface, likely VS Code, showing a Python file named 'app.py'. The code defines a Flask application with a single route '/status' that returns a JSON response. The application is configured to run in debug mode. The interface includes a sidebar with 'EXPLORER' and 'OPEN EDITORS' views, and a main editor area with a dark theme.

```
1 from flask import Flask, jsonify
2
3 app = Flask(__name__)
4
5 @app.route('/status', methods=['GET'])
6 def status():
7     return jsonify({"status": "API is running"})
8
9 if __name__ == '__main__':
10     app.run(debug=True)
```

Output:

A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:5000/status'. Below the address bar, there is a 'Pretty-print' button. The main content area displays a JSON object:

```
{  
  "status": "API is running"  
}
```

Observation

The Flask server started successfully on `http://127.0.0.1:5000/` without any errors.

1. Accessing the endpoint `http://127.0.0.1:5000/status` returned the expected JSON response:

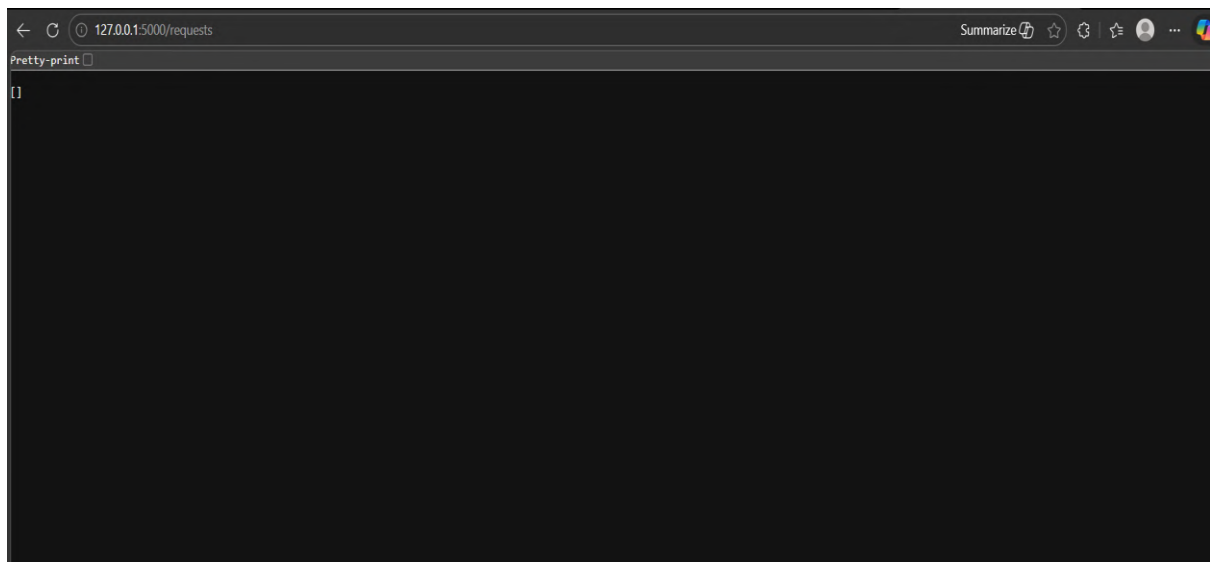
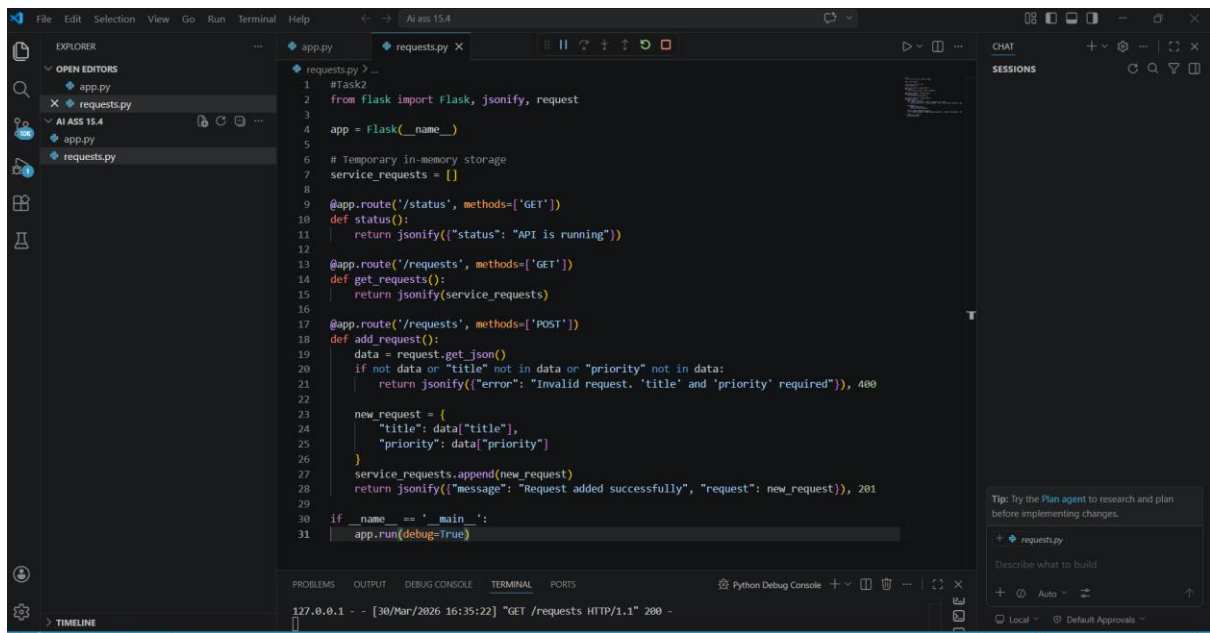
```
{"status": "API is running"}
```
2. The response was verified using both a browser and curl, and the output matched the expected result.

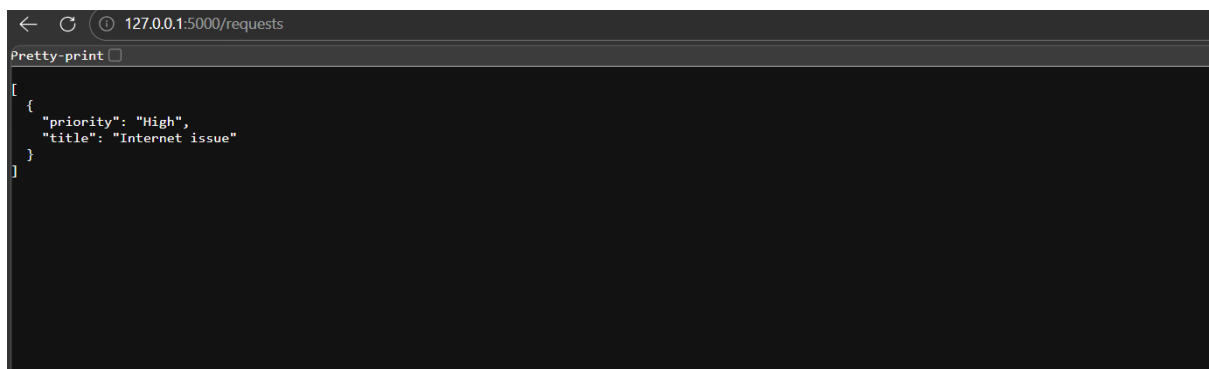
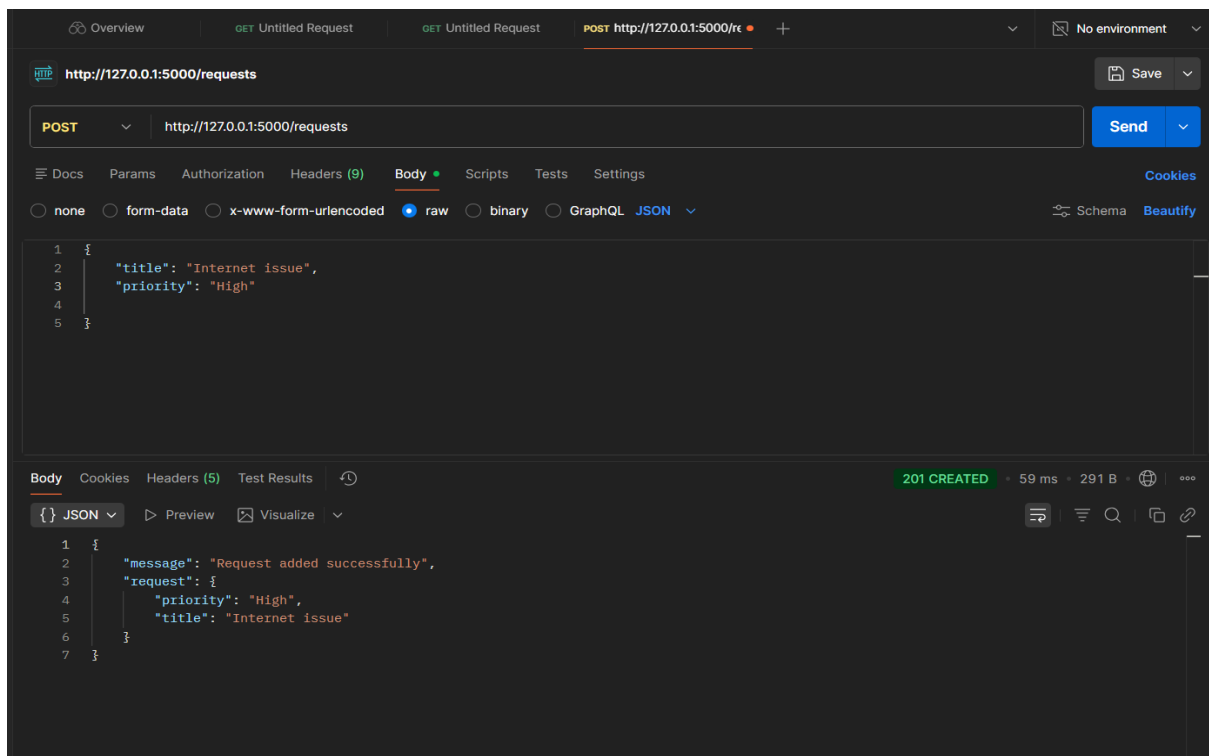
Task2:

Prompt:

Create a GET endpoint `/requests` to retrieve all service requests stored in memory.

1. Create a POST endpoint `/requests` to add a new request with title and priority.
2. Verify that GET initially returns an empty list.
3. Use POST to add a request and confirm the response.
4. Run GET again to ensure the added request appears in the list.





Observation:

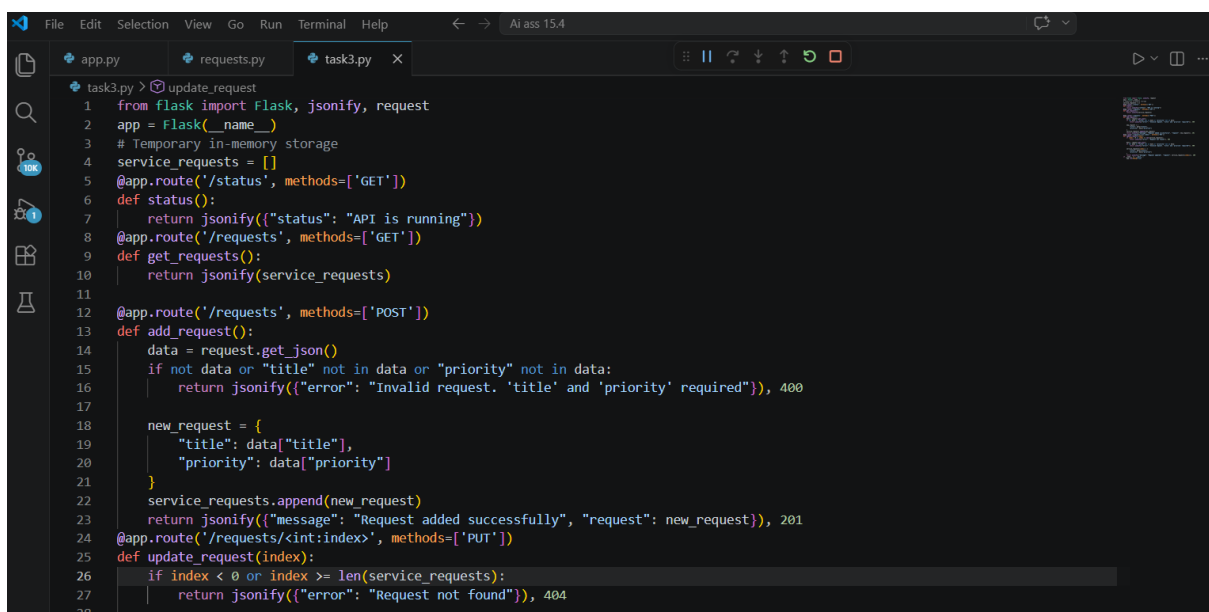
1. GET `/requests` initially returned an empty list `[]`, confirming no requests were stored.
2. POST `/requests` with `{"title": "Internet issue", "priority": "High"}` returned a success message along with the added request object.
3. Subsequent GET `/requests` returned the list containing the newly added request, verifying that the POST and GET endpoints are working correctly.

Task3:

Prompt:

Create a PUT endpoint `/requests/` to update a service request by its index.

1. Ensure the request body contains both title and priority.
2. Return the updated request object along with a success message.
3. Handle invalid indexes gracefully by returning a clear error message with HTTP 404.
4. Verify that subsequent GET requests reflect the updated data.

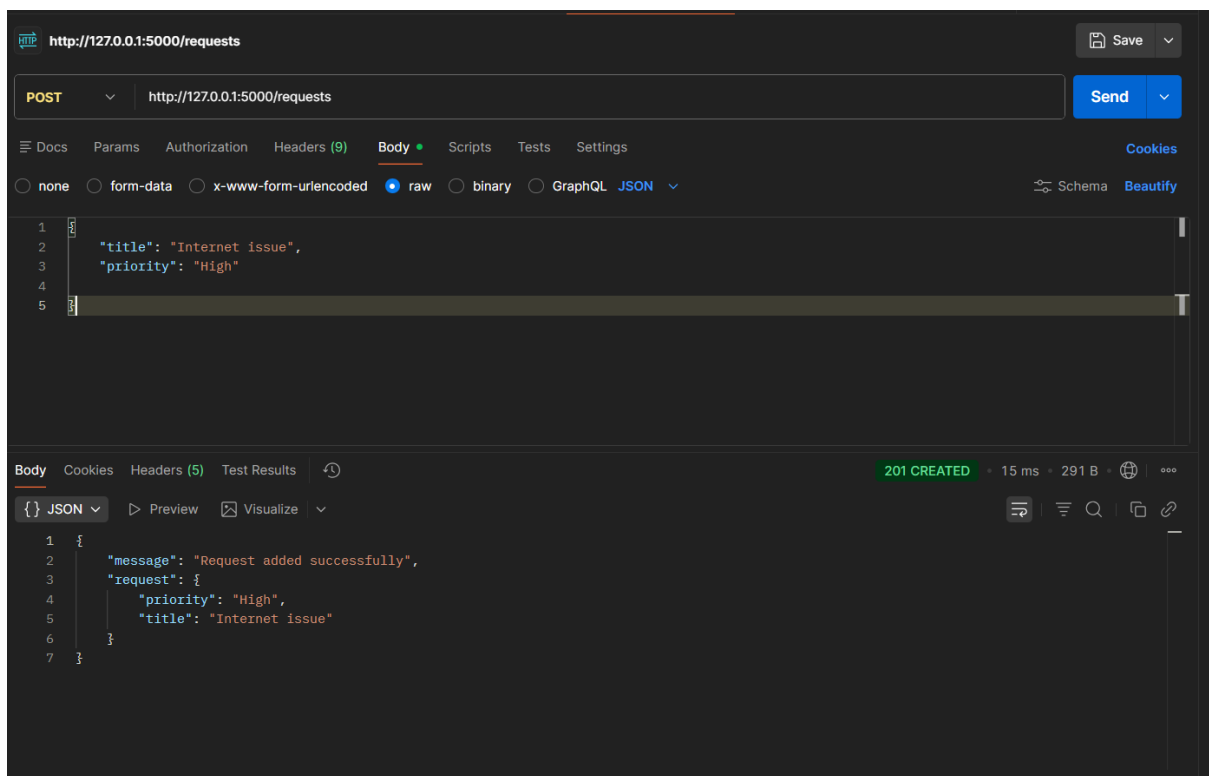


```
task3.py > update_request
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3 # Temporary in-memory storage
4 service_requests = []
5 @app.route('/status', methods=['GET'])
6 def status():
7     return jsonify({"status": "API is running"})
8 @app.route('/requests', methods=['GET'])
9 def get_requests():
10     return jsonify(service_requests)
11
12 @app.route('/requests', methods=['POST'])
13 def add_request():
14     data = request.get_json()
15     if not data or "title" not in data or "priority" not in data:
16         return jsonify({"error": "Invalid request. 'title' and 'priority' required"}), 400
17
18     new_request = {
19         "title": data["title"],
20         "priority": data["priority"]
21     }
22     service_requests.append(new_request)
23     return jsonify({"message": "Request added successfully", "request": new_request}), 201
24 @app.route('/requests/<int:index>', methods=['PUT'])
25 def update_request(index):
26     if index < 0 or index >= len(service_requests):
27         return jsonify({"error": "Request not found"}), 404
28
```

```
File Edit Selection View Go Run Terminal Help  ← →  Ai ass 15.4

task3.py  requests.py  task3.py  X

task3.py > update_request
13 def add_request():
17
18     new_request = {
19         "title": data["title"],
20         "priority": data["priority"]
21     }
22     service_requests.append(new_request)
23     return jsonify({"message": "Request added successfully", "request": new_request}), 201
24 @app.route('/requests/<int:index>', methods=['PUT'])
25 def update_request(index):
26     if index < 0 or index >= len(service_requests):
27         return jsonify({"error": "Request not found"}), 404
28
29     data = request.get_json()
30     if not data or "title" not in data or "priority" not in data:
31         return jsonify({"error": "Invalid request. 'title' and 'priority' required"}), 400
32
33     service_requests[index] = {
34         "title": data["title"],
35         "priority": data["priority"]
36     }
37     return jsonify({"message": "Request updated", "request": service_requests[index]}), 200
38 if __name__ == '__main__':
39     app.run(debug=True)
```



http://127.0.0.1:5000/requests/0

PUT http://127.0.0.1:5000/requests/0 Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "title": "Internet issue fixed",
3   "priority": "Low"
4 }
5 }
```

Body Cookies Headers (5) Test Results 200 OK • 31 ms • 280 B

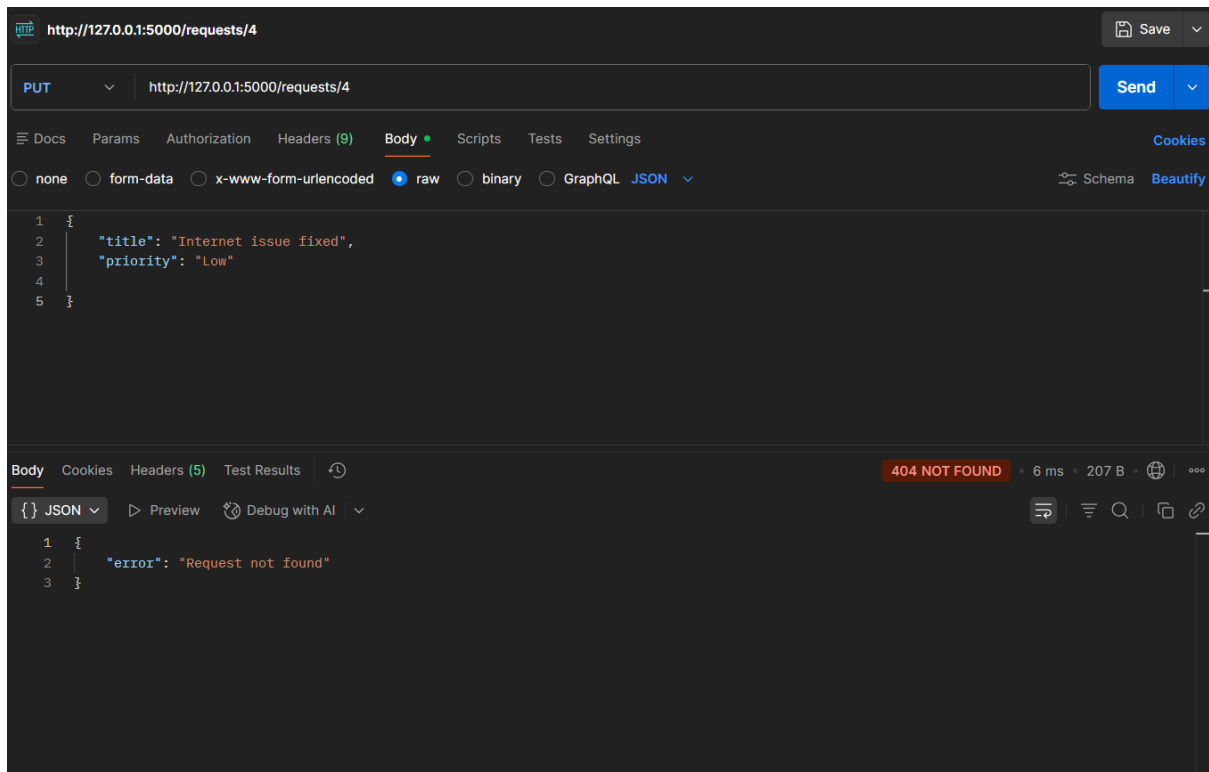
{ JSON Preview Visualize

```
1 {
2   "message": "Request updated",
3   "request": {
4     "priority": "Low",
5     "title": "Internet issue fixed"
6   }
7 }
```

127.0.0.1:5000/requests

pretty-print

```
{
  "priority": "Low",
  "title": "Internet issue fixed"
}
```



Observation:

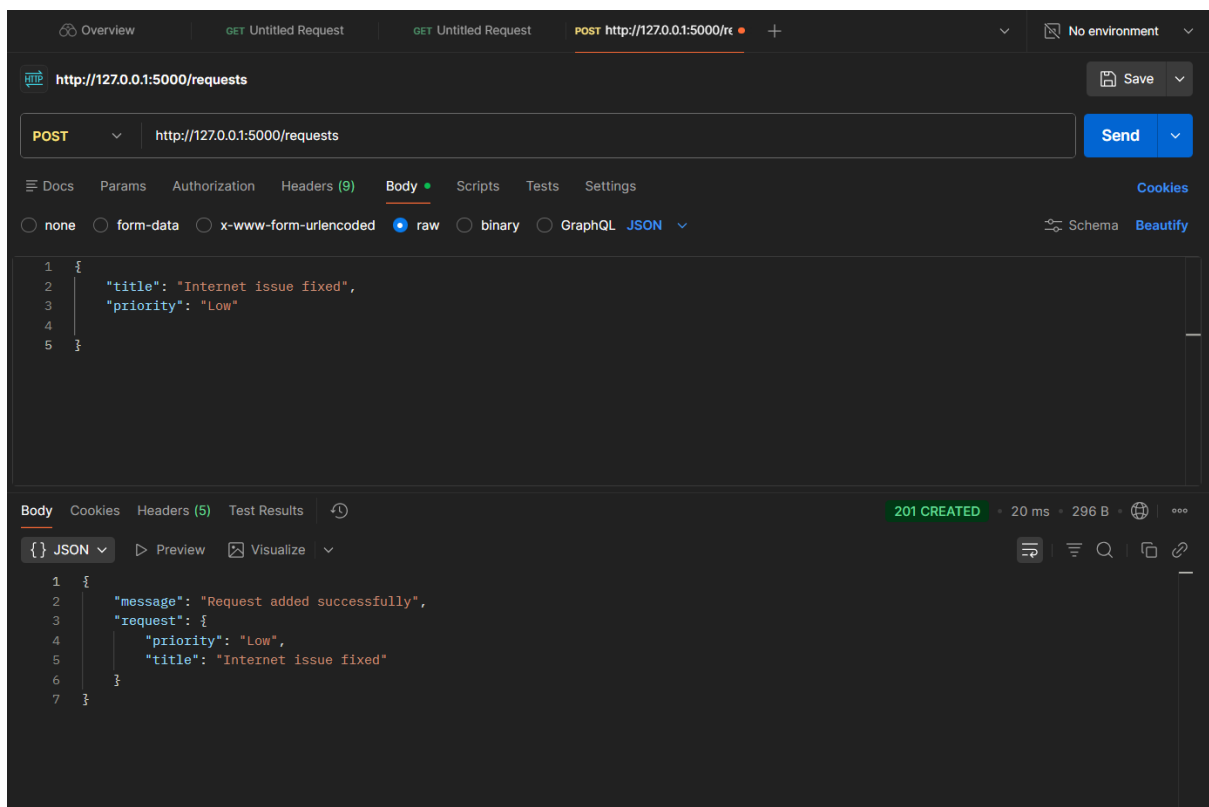
1. A valid PUT /requests/0 successfully updated the request's priority and returned the modified object with the message: `{"message": "Request updated", "request": {...}}`
2. An invalid PUT /requests/5 returned a clear error response: `{"error": "Request not found"}`
3. with HTTP 404, confirming proper error handling.
4. The updated request was correctly reflected in subsequent GET /requests calls, proving that the PUT endpoint works as intended and integrates seamlessly with the existing GET and POST endpoints.

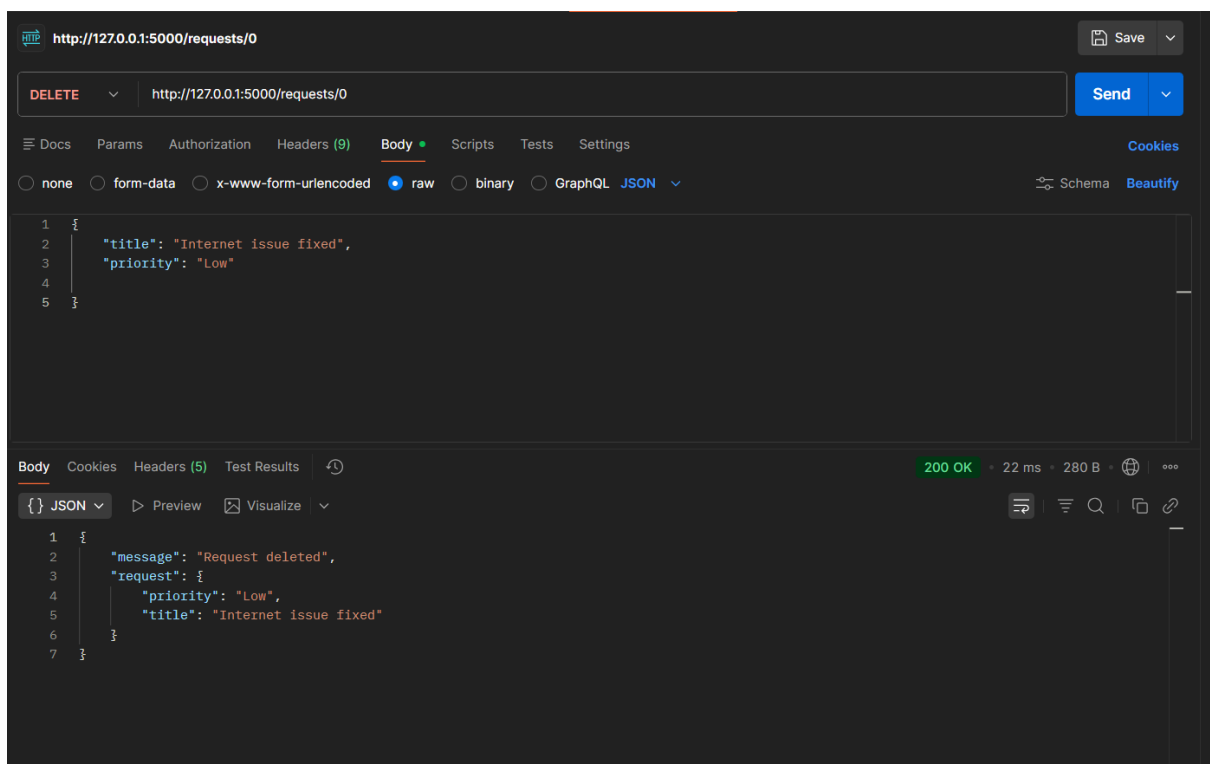
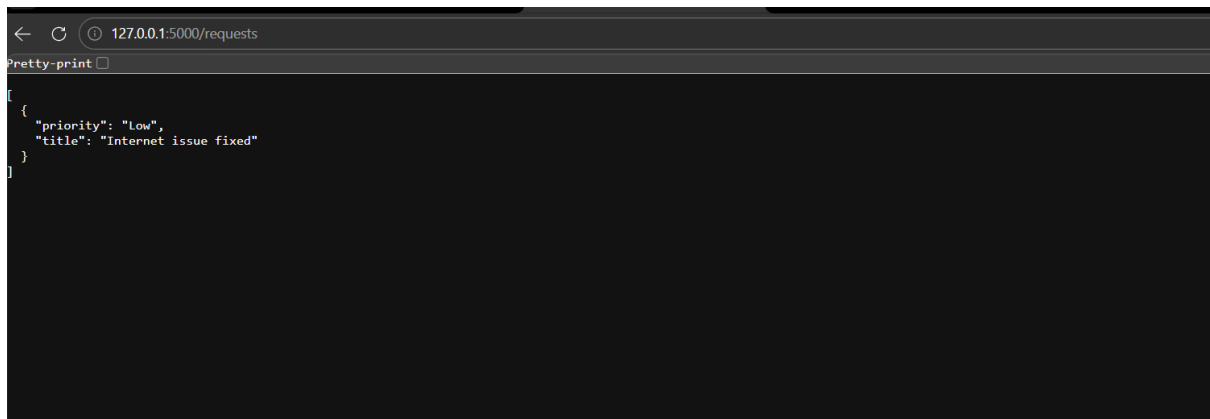
Task4:

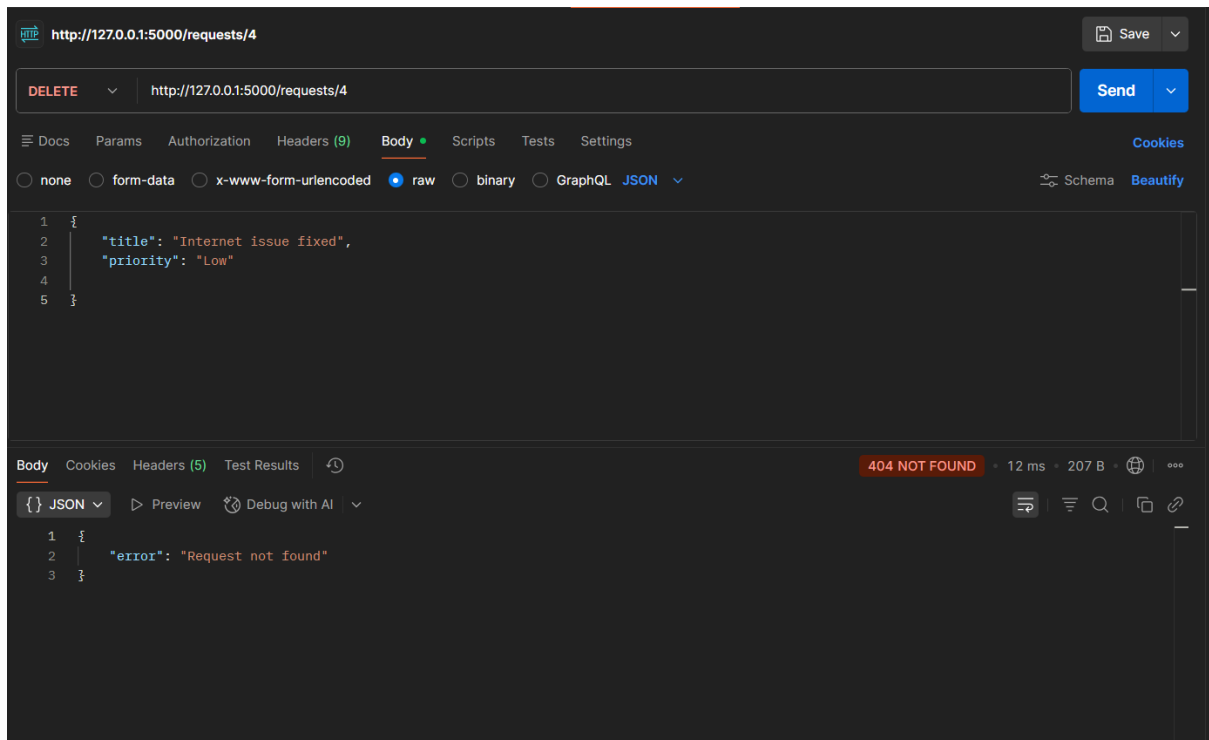
Prompt:

Create a DELETE endpoint `/requests/` to remove a service request by its index.

1. Ensure the deleted request details are returned in the response for confirmation.
2. Handle invalid indexes gracefully by returning a clear error message with HTTP 404.
3. Verify that subsequent GET requests confirm the removal of the deleted request.







Observation

1. A valid DELETE /requests/0 successfully removed the request and returned its details with a confirmation message:

```
{"message": "Request deleted", "request": {...}}
```

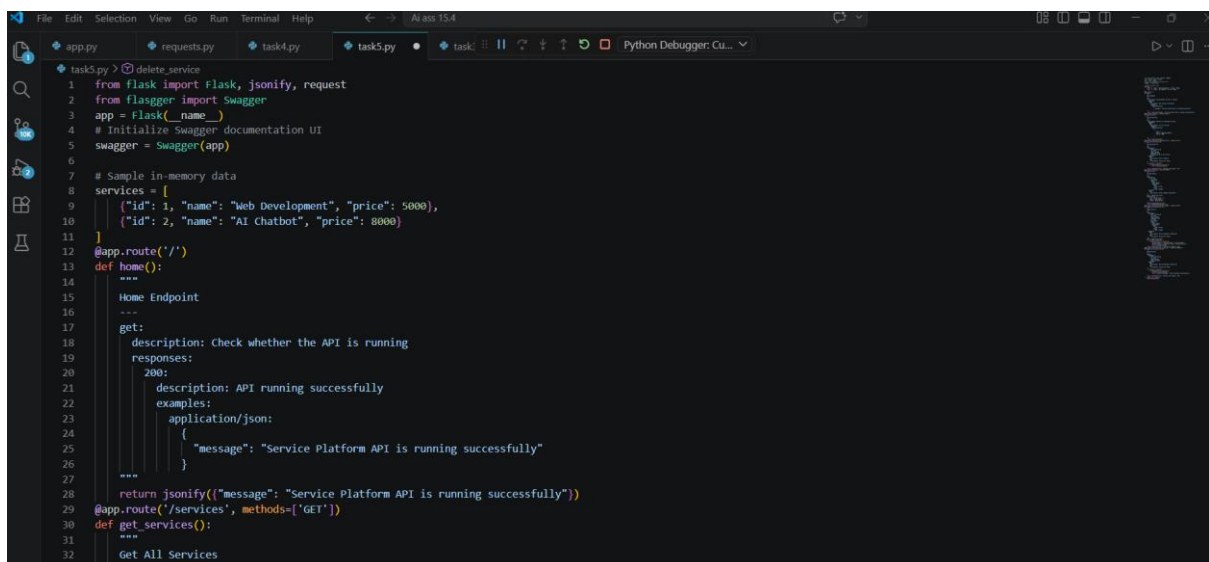
2. An invalid DELETE /requests/5 returned a clear error message with HTTP 404, confirming safe error handling without crashing the server.
3. Subsequent GET /requests calls showed the list without the deleted request, proving that the DELETE endpoint works correctly and integrates seamlessly with the existing CRUD operations.

Task5:

Prompt

Add meaningful docstrings to all endpoints describing their purpose, HTTP method, inputs, and outputs.

1. Insert inline comments in the code to make logic and flow easy to follow.
2. Clearly describe request methods, expected JSON body, and sample responses for each endpoint.
3. Optionally integrate Swagger/Flask-RESTX to automatically generate interactive API documentation showing:
 - Endpoint paths
 - HTTP methods
 - Sample request/response formats

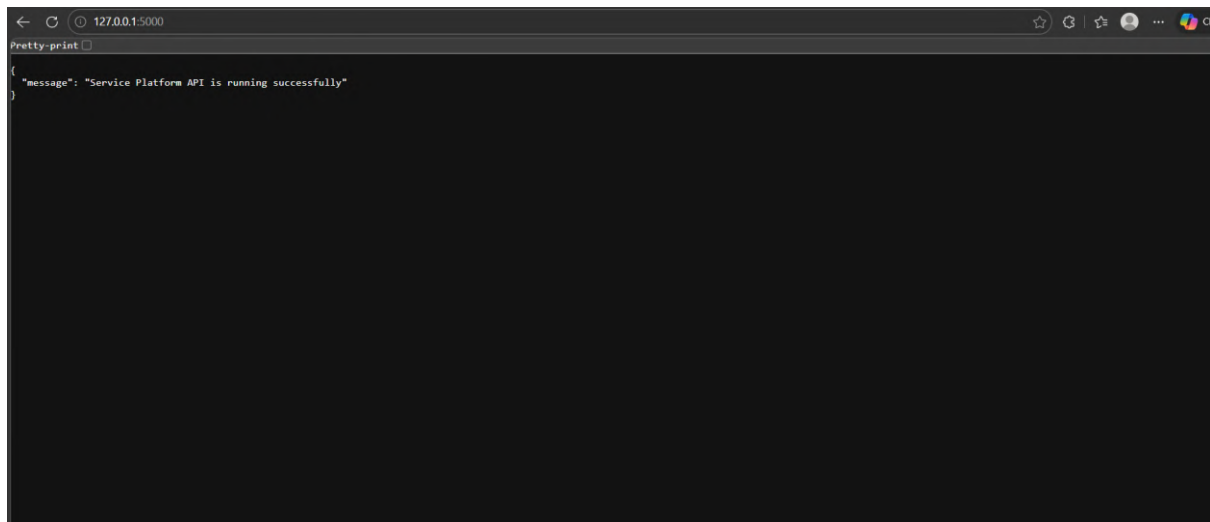


```
File Edit Selection View Go Run Terminal Help
task5.py > delete_service
1 from flask import Flask, jsonify, request
2 from flask_swagger import Swagger
3 app = Flask(__name__)
4 # Initialize Swagger documentation UI
5 swagger = Swagger(app)
6
7 # Sample in-memory data
8 services = [
9     {"id": 1, "name": "Web Development", "price": 5000},
10    {"id": 2, "name": "AI Chatbot", "price": 8000}
11]
12
13@app.route('/')
14def home():
15    """
16    Home Endpoint
17    """
18    get:
19        description: Check whether the API is running
20        responses:
21            200:
22                description: API running successfully
23                examples:
24                    application/json:
25                        {
26                            "message": "Service Platform API is running successfully"
27                        }
28    return jsonify({"message": "Service Platform API is running successfully"})
29
30@app.route('/services', methods=['GET'])
31def get_services():
32    """
33    Get All Services
34    """
```

```
app.py requests.py task4.py task5.py task1 Python Debugger: Cu...
task5.py > delete_service
30 def get_services():
31     """
32     Get All Services
33     """
34     get:
35     description: Returns all available services
36     responses:
37     200:
38     description: List of services
39     examples:
40     application/json:
41     [
42     {
43     "id": 1,
44     "name": "Web Development",
45     "price": 5000
46     }
47     ]
48     """
49     return jsonify(services)
50 @app.route('/services/<int:service_id>', methods=['GET'])
51 def get_service(service_id):
52     """
53     Get Service By ID
54     """
55     get:
56     parameters:
57     - name: service_id
58       in: path
59       type: integer
60       required: true
61     description: ID of the service
62     responses:
```

```
app.py requests.py task4.py task5.py task1 Python Debugger: Cu...
task5.py > delete_service
51 def get_service(service_id):
65     """
66     404:
67     description: Service not found
68     """
69     for service in services:
70     if service["id"] == service_id:
71     return jsonify(service)
72     return jsonify({"error": "Service not found"}), 404
73 @app.route('/services', methods=['POST'])
74 def add_service():
75     """
76     Add New Service
77     """
78     post:
79     parameters:
80     - in: body
81       name: body
82       schema:
83       type: object
84       properties:
85       name:
86       type: string
87       price:
88       type: integer
89     responses:
90     201:
91     description: Service added successfully
92     """
93     data = request.get_json()
94     new_service = {
95     "id": len(services) + 1,
```

```
app.py requests.py task4.py task5.py task1 Python Debugger: Cu...
task5.py > delete_service
102 def update_service(service_id):
103     """
104     Update Service
105     """
106     if service["id"] == service_id:
107     service["name"] = data.get("name", service["name"])
108     service["price"] = data.get("price", service["price"])
109     return jsonify(service)
110     return jsonify({"error": "Service not found"}), 404
111 @app.route('/services/<int:service_id>', methods=['DELETE'])
112 def delete_service(service_id):
113     """
114     Delete Service
115     """
116     delete:
117     parameters:
118     - name: service_id
119       in: path
120       type: integer
121       required: true
122     responses:
123     200:
124     description: Service deleted successfully
125     404:
126     description: Service not found
127     """
128     for service in services:
129     if service["id"] == service_id:
130     services.remove(service)
131     return jsonify({"message": "Service deleted successfully"})
132     return jsonify({"error": "Service not found"}), 404
133 if __name__ == '__main__':
134     app.run(debug=True)
```



Observation:

Each endpoint now has well-structured docstrings that explain its purpose, request method, required inputs, and possible responses, making the API self-explanatory.

1. Inline comments improve readability, allowing another developer to quickly understand the logic without reading the full code.
2. The API is ready for optional integration with Swagger/Flask-RESTX, which would provide a browser-based documentation UI displaying endpoints, methods, and sample request/response formats for easier collaboration.